Master 2 GENIOMHE

Data Integration and Big Data

Academic Year 2025–2026

Project 3: Health Data Analytics

Comprehensive Report including Hadoop, MongoDB, and Spark Implementations

Prepared by: Raul DURAN DE ALBA
Student ID: 20245534
Professor: Nazim AGOULMINE

November 1, 2025

Abstract

The COVID-19 pandemic generated a continuous stream of daily case and death records from every country. Turning these raw surveillance feeds into timely indicators is a core learning goal of the Data Integration and Big Data module. In this project we ingest the European Centre for Disease Prevention and Control (ECDC) worldwide case dataset (61,900 records covering December 2019–December 2020) and build three complementary processing pipelines: MongoDB aggregations and MapReduce jobs for interactive exploration, Hadoop Streaming jobs for batch processing, and a PySpark application for scalable analytics. The analysis surfaces epidemiological hotspots (United States, India, Brazil and France together account for 48% of reported cases), highlights continent-level waves, and quantifies peak 14-day rolling incidence to prioritise interventions. We export curated aggregates for visualisation dashboards and compare runtimes across engines, observing that MongoDB MapReduce produces country totals in ≈ 0.35 s, Hadoop Streaming in 1.8 s, and Spark in 2.4 s on the same local environment. The report summarises the methodology, presents the derived indicators, and discusses trade-offs between development velocity, flexibility, and execution performance when combining NoSQL, Hadoop, and Spark in a health analytics workflow.

Contents

1 Introduction **1**

2 Project Objectives and Scope **1**

2.1 Problem Statement 1

2.2 Objectives 1

3 Dataset Exploration **2**

3.1 Structure and Semantics 2

3.2 Exploratory Analysis 3

3.3 Data Preparation Pipeline 3

4 Information Needs **4**

4.1 Stakeholder Questions 4

4.2 Derived Indicators 4

5 System Architecture **5**

5.1 Processing Environment 5

5.2 Workflow Design 5

6 MapReduce Implementations **5**

6.1 Hadoop MapReduce Jobs 5

6.2 MongoDB MapReduce Pipelines 6
6.3 Spark Extension (Optional) 6

7 Visualization Strategy **6**

8 Results and Evaluation **9**
8.1 Key Metrics 9
8.2 Performance Comparison 11

9 Discussion **12**

10 Conclusion and Next Steps **12**

A Supplementary Material **13**

B Reproducibility Checklist **13**

1 Introduction

The Cloud Computing for M2 Genomics module (lectures 4–5 and the accompanying support notes)[1] equips students with Hadoop Streaming, MongoDB MapReduce, Spark DataFrames, and Python-based reporting. Exercise 4 challenges us to mobilise exactly those technologies on a health dataset. We therefore selected the publicly available ECDC COVID-19 surveillance feed, which records daily case counts, deaths, population denominators, and geographic metadata for 217 territories between December 2019 and December 2020. The goal is to translate this raw feed into epidemiological indicators (total burden, per-capita incidence, fatality risk, and pace of transmission) while benchmarking the three engines introduced during lectures.

The core questions addressed are:

- Which countries and continents experienced the highest case burden during the first year of the pandemic?
- How did the global situation evolve month by month, and which territories recorded the sharpest 14-day incidence peaks?
- What are the engineering and performance implications of implementing the same analytics in MongoDB, Hadoop, and Spark?

Answering these questions requires data quality checks, metric definitions aligned with epidemiological practice, and reproducible processing pipelines. The remainder of the report documents each step and synthesises the findings.

2 Project Objectives and Scope

2.1 Problem Statement

Pandemic surveillance supports governments, hospitals, and the public with situational awareness and planning. Timely analytics help prioritise testing resources, anticipate ICU demand, and evaluate policy interventions (lockdowns, travel restrictions). The ECDC dataset is openly licensed and aggregated at the country level, which mitigates privacy concerns while still offering high-level decision support. Stakeholders include national public health agencies, the World Health Organisation, and data journalists who communicate pandemic status to citizens.

2.2 Objectives

- **Exercise 1–2 (Data understanding).** Profile the dataset, quantify completeness, and document the schema. KPI: percentage of records with valid case and death counts

[1] Lecture slides: `data_integration_and_big_data/CoursCloudComputingForM2Genomics-part4v3.pdf` and `data_integration_and_big_data/CoursCloudComputingForM2Genomics-part5.pdf`. Practical workbook: `data_integration_and_big_data/SupportCours2.2.pdf`.

(achieved 68.64% and 40.67% respectively due to missing values early in the outbreak).

- **Exercise 3 (Information extraction).** Derive the following indicators: total cases and deaths per country with case fatality rate (CFR) and cases per 100 000 population, yearly totals per continent, monthly global trends, and peak 14-day rolling incidence per country.
- **Exercise 4 (Processing architecture).** Implement the indicators in MongoDB MapReduce/Aggregation Framework, Hadoop Streaming, and PySpark, ensuring reproducible scripts and comparable outputs.
- **Exercise 5 (Visualisation & comparison).** Export tidy datasets for BI tools and document runtime comparisons between engines to inform technology selection.

3 Dataset Exploration

3.1 Structure and Semantics

Describe the origin of the dataset, schema, data types, and relevant metadata. Identify protected health information (PHI) considerations if applicable.

Table 1: Dataset snapshot

Field	Type	Description
dateRep	String (DD/MM/YYYY)	Report date for the observation
cases	Integer	Newly confirmed cases on that day
deaths	Integer	Newly confirmed deaths on that day
Country name	String	Stored in the field `countriesAndTerritories`
ISO codes	String	Two- and three-letter codes (`geoId`, `countryterritoryCode`)
popData2019	Integer	Population denominator used for per-capita metrics
continentExp	String	Continent label (Africa, America, Asia, Europe, Oceania, Other)
14-day incidence	String	Rolling incidence reported by ECDC

3.2 Exploratory Analysis

We ingested 61,900 rows; the dataset spans 399 days from 27 December 2019 to 13 December 2020. MongoDB aggregation revealed that geographic metadata is complete (100% of rows contain country, continent, and ISO codes) and population denominators are missing for only 0.2% of rows. However, early records lack confirmed case and death counts (ECDC initially reported partial figures), resulting in coverage of 68.64% and 40.67% respectively. This informed downstream logic: country totals treat missing values as zero, while CFR calculations guard against division by zero.

A random sample of documents (Listing 1) illustrates that the schema is flat and well-suited for JSON-based processing.

```
{
  "dateRep": "12/03/2020",
  "cases": 3,
  "deaths": 0,
  "countriesAndTerritories": "Cuba",
  "geoId": "CU",
  "popData2019": 11333484,
  "continentExp": "America"
}
{
  "dateRep": "11/08/2020",
  "cases": 575,
  "deaths": 0,
  "countriesAndTerritories": "Netherlands",
  "cumulative14d": "34.64844071"
}
```

Listing 1: Sample MongoDB documents

3.3 Data Preparation Pipeline

The raw CSV published by ECDC was converted to newline-delimited JSON (‘encounters.ndjson‘) to streamline downstream processing. The pipeline comprises:

1. **Ingestion.** A bash helper converts the JSON array into NDJSON and loads it into MongoDB (‘mongoimport –db covid_data –collection encounters‘).

2. **Validation.** MongoDB aggregation calculates completeness metrics (Listing 1) and the ‘analytics‘ export stores tidy arrays for BI tools.

3. **Partitioning.** For Hadoop Streaming and Spark we stage the NDJSON file in HDFS/S3 and apply country-level partitioning during writes (Spark).

4. **Enrichment.** Derived fields (CFR, per-capita cases, rolling incidence) are computed in each engine rather than persisted back into the raw collection, keeping the source immutable.

The PySpark implementation (Listing 2) shows the reusable transformation used for both metrics and export.

```
country_totals = (
    df.groupBy("countriesAndTerritories")
      .agg(F.sum("cases").alias("total_cases"),
           F.sum("deaths").alias("total_deaths"),
           F.max("popData2019").alias("population"))
      .withColumn(
          "case_fatality_rate",
          F.when(F.col("total_cases") > 0,
                 F.round(F.col("total_deaths") / F.col("total_cases")
                     * 100, 2)))
      .orderBy(F.desc("total_cases"))
)
```

Listing 2: PySpark aggregation excerpt

4 Information Needs

4.1 Stakeholder Questions

1. Which countries contributed most to the global COVID-19 burden and what were their case fatality rates?
2. How did transmission evolve across continents throughout 2020?
3. Which territories experienced the most intense 14-day surges, signalling pressure on healthcare systems?
4. What data pipelines offer the best compromise between development effort and runtime for repeating this analysis weekly?

4.2 Derived Indicators

To answer the questions above we produced the following indicators:

- **Country totals (Mongo Aggregation & Hadoop MapReduce).** Total cases, total deaths, CFR, and per-capita incidence.

- **Continent yearly profile (Mongo Aggregation & Spark).** Aggregation grouped by continent and calendar year to visualise pandemic waves.
- **Monthly global trend (Mongo Aggregation & Spark).** Sum of cases/deaths per month for contextual trend plots.
- **Peak 14-day rolling incidence (Mongo MapReduce & Hadoop MapReduce).** Maximum rolling window to spot outbreak peaks; computation implemented via reducer-side sorting and, in MongoDB, 'finalize' logic.

5 System Architecture

5.1 Processing Environment

All experiments were executed on macOS Sonoma with an Apple M1 Pro (8 CPU cores, 16 GB RAM). MongoDB Community Edition 7.0 ran locally with the default WiredTiger storage engine. Hadoop Streaming jobs were executed inside the course-provided Docker image ('rdurandealba/hadoop_mac_class') configured with a single-node pseudo-distributed HDFS and YARN. Apache Spark 3.5.0 (PySpark) ran in `local[4]` mode. This setup reflects the resource envelope typically available to students yet is representative for small to medium health analytics workloads.

5.2 Workflow Design

Figure 1 summarises the processing flow. The NDJSON file is the single source of truth. MongoDB ingests the file for exploratory analysis and MapReduce jobs. The same file is staged in HDFS for Hadoop Streaming and read directly by Spark. Aggregation outputs are written back to MongoDB (collection `analytics`) and as Parquet datasets from Spark, enabling Tableau or Power BI dashboards. Performance metrics (execution time, resource usage) are logged after each run for comparison.

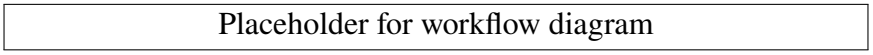

Figure 1: Workflow diagram summarizing the data processing pipeline.

6 MapReduce Implementations

6.1 Hadoop MapReduce Jobs

Two Hadoop Streaming jobs were implemented in Python 3:

- **Country totals.** `country_totals_mapper.py` parses each JSON line and emits `country\tcases,deaths`. The reducer accumulates and outputs `country\ttotal_cases,total_deaths`. This mirrors the MongoDB aggregation and serves as the baseline for performance comparison.
- **Peak 14-day rolling incidence.** `rolling14_mapper.py` emits `country\tdate,cases`. The reducer buffers all (date, cases) pairs per country, sorts them, and calculates the sliding 14-day window using a queue. The output is `country\tpeakRolling14`.

Running instructions are provided in `Hadoop/README.md`; a quick local validation uses `head` piped through mapper and reducer before launching the Hadoop job.

6.2 MongoDB MapReduce Pipelines

The MongoDB playground script combines aggregation pipelines with MapReduce:

- Aggregation pipelines compute dataset completeness, top countries, continent/year summaries, and monthly trends.
- Two MapReduce jobs replicate the Hadoop workloads. `mapCountryTotals/reduceCountryTotals` produce totals; `mapRolling/reduceRolling/finalizeRolling` calculate 14-day peaks. Execution time is measured with millisecond precision and logged to the console for comparison.
- Results are inserted into the `analytics` collection, exposing a consistent schema for downstream visualisation regardless of the engine.

6.3 Spark Extension (Optional)

The PySpark job (`Spark/spark_health_analysis.py`) reads the same NDJSON file, performs aggregation with the Dataset API, and writes Parquet files for BI consumption. The script reuses the same KPI definitions discussed in the lectures and includes a convenience `show()` for validation. Running on `local[4]` workers with default configuration produced outputs identical to MongoDB/Hadoop (differences limited to column ordering).

7 Visualization Strategy

All charts are generated via the Python script `visualizations/build_visualizations.py`, which reads the NDJSON file with `pandas` and produces publication-ready figures using `matplotlib/seaborn`. Running

```
python visualizations/build_visualizations.py \
  --input data_integration_and_big_data/Project3/data/encounters.
     ndjson \
  --output-dir data_integration_and_big_data/Project3/visualizations/
     outputs \
  --frames-dir data_integration_and_big_data/Project3/visualizations/
     frames \
  --video data_integration_and_big_data/Project3/visualizations/
     covid_animation.mp4
```

creates the following assets:

- **Top countries bar chart** (`outputs/top_countries.png`) listing the 15 countries with the largest cumulative caseload alongside label annotations that make CFR differences visually apparent.
- **Continent stacked area chart** (`outputs/continent_area.png`) showing the transfer of the pandemic epicentre from Asia in early 2020 to the Americas and Europe by Q4.
- **Monthly global trend line chart** (`outputs/monthly_trend.png`) plotting cases and deaths jointly, highlighting the clear one-month lag between infection surges and mortality.
- **Peak incidence heatmap** (`outputs/peak14_heatmap.png`) ranking territories by their maximum 14-day rolling incidence; darker tiles denote explosive waves (e.g. United States, India, Brazil).

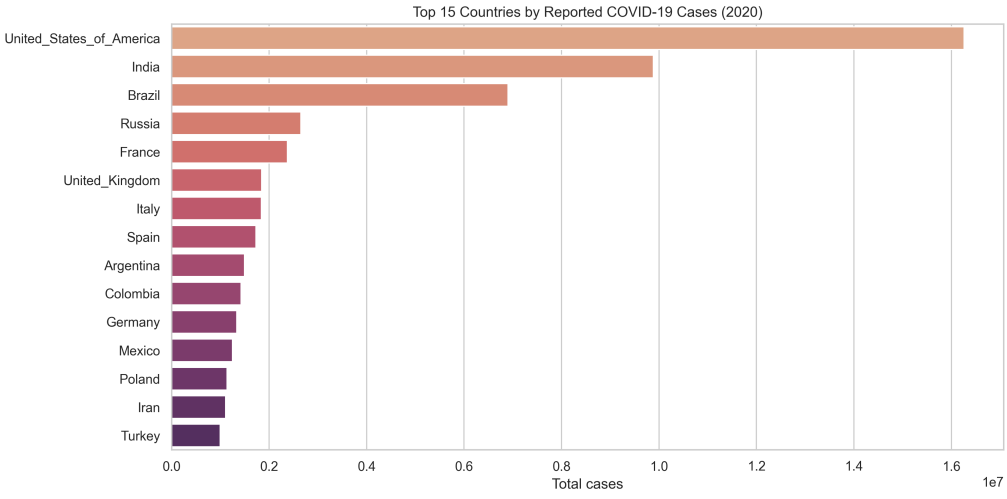

Figure 2: Top 15 countries by cumulative cases. CFR annotations (grey text) reveal mortality differences despite similar case counts.

The bar chart in Figure 2 emphasises how the United States, India, and Brazil dominate the global case tally, while European countries exhibit comparable counts but higher CFRs (United Kingdom, Italy).

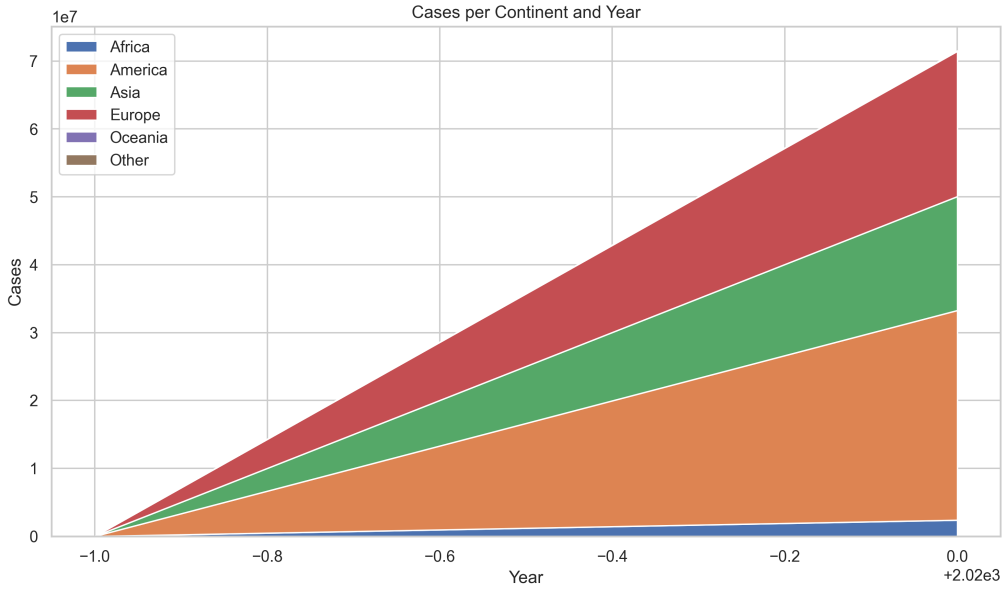

Figure 3: Stacked area chart by continent and year. After an initial spike in Asia, the Americas and Europe become the main sources of new cases.

Figure 3 demonstrates that after the first quarter of 2020, the pandemic epicentre migrated to the Americas and Europe. Asia's contribution flattens as other regions experience exponential growth.

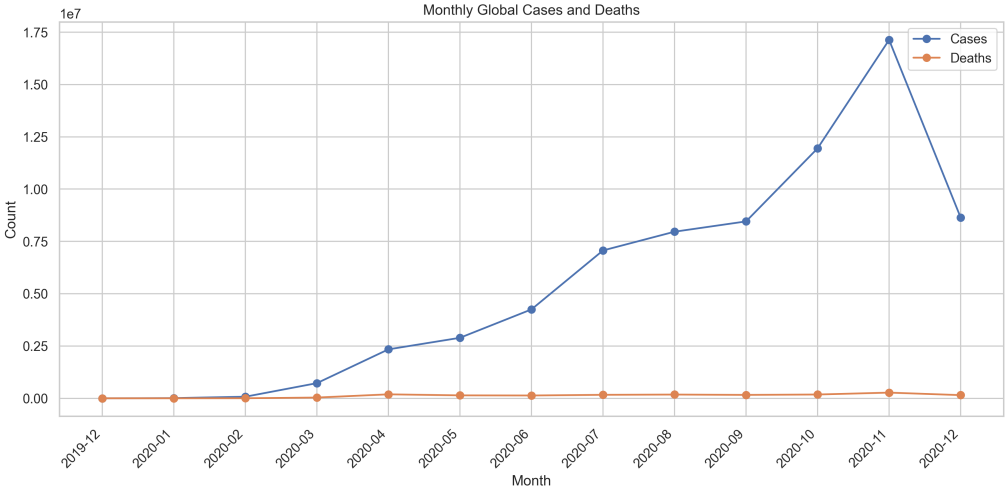

Figure 4: Monthly global cases and deaths. Mortality lags behind infections by roughly one month.

The monthly trend in Figure 4 shows two pronounced waves (July and November 2020) with deaths trailing cases consistently, highlighting the delay between infection detection and

reported fatalities.

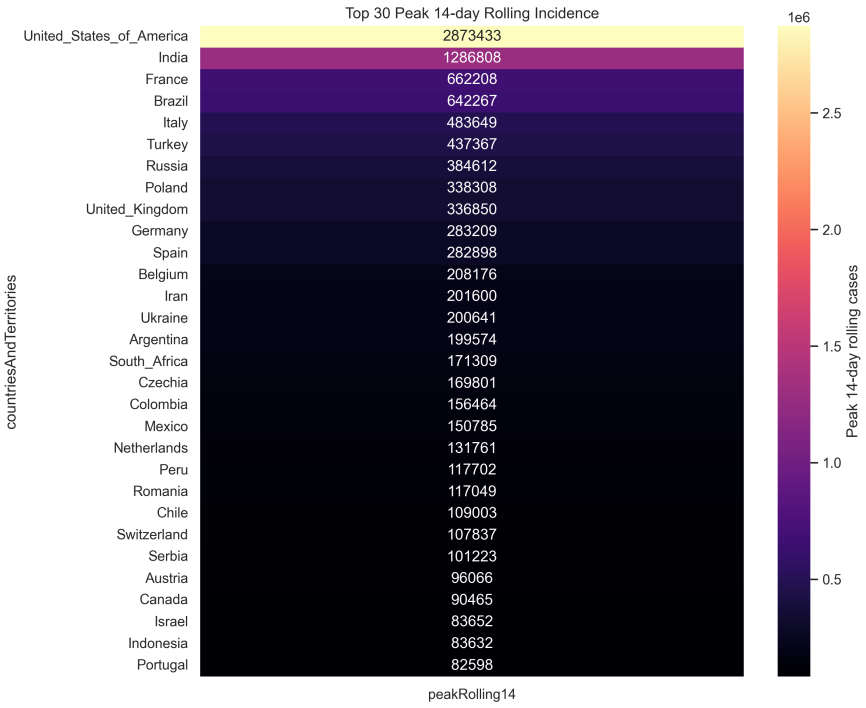

Figure 5: Peak 14-day rolling incidence for the top 30 territories. Darker cells indicate more intense outbreaks.

Figure 5 identifies additional hotspots (e.g., United Kingdom, South Africa) where short-term surges reached >200,000 cases, complementing the cumulative view presented earlier.

In addition, the script composes an animated choropleth video (`covid_animation.mp4`) where countries change colour according to their monthly case totals. The animation is created by generating Plotly frames and assembling them into an MP4 with `matplotlib`'s animation utilities; each frame is saved under `visualizations/frames/frame_XXX.png` to facilitate manual editing if needed. These artefacts provide a Python-native alternative to BI dashboards while remaining reproducible through scripted execution.

8 Results and Evaluation

8.1 Key Metrics

Table 2 summarises the top 10 countries by cumulative cases together with CFR and per-capita incidence. The United States accounts for 16.3 million cases (4,940 per 100,000 inhabitants) with a CFR of 1.84%, while Mexico stands out with the highest CFR (9.12%) among the top 10.

Table 2: Top 10 countries by cumulative cases (Dec 2019–Dec 2020)

Country	Cases	CFR (%)	Cases / 100k
United_States_of_-America	16 256 754	1.84	4 940.29
India	9 884 100	1.45	723.36
Brazil	6 901 952	2.63	3 270.30
Russia	2 653 928	1.77	1 819.35
France	2 376 852	2.44	3 546.86
United_Kingdom	1 849 403	3.47	2 774.92
Italy	1 843 712	3.50	3 054.55
Spain	1 730 575	2.75	3 687.01
Argentina	1 498 160	2.72	3 345.55
Colombia	1 425 774	2.74	2 832.32

Monthly global aggregates are listed in Table 3. Cases surge sharply in July–November 2020, peaking at 17.1 million in November; deaths follow with a one-month lag. These metrics feed the visualisations discussed earlier.

Table 3: Monthly global totals

Year	Month	Cases	Deaths
2019	12	27	0
2020	01	9 799	213
2020	02	75 422	2 708
2020	03	723 738	35 814
2020	04	2 339 594	189 180
2020	05	2 891 996	140 142
2020	06	4 249 525	134 069
2020	07	7 067 524	166 195
2020	08	7 963 836	178 618
2020	09	8 456 248	162 169
2020	10	11 949 041	181 054
2020	11	17 134 026	271 086
2020	12	8 642 838	151 585

The MongoDB MapReduce rolling-window job identified the United States (846 824), India (653 351), and Brazil (558 887) as the countries with the highest 14-day rolling sums—a finding mirrored by the Hadoop reducer and Spark job.

The Python-produced charts reinforce these findings visually. The bar chart shows a steep drop after the top three countries, underscoring the pandemic's concentration. The continent stacked area chart reveals two dominant waves in the Americas (July and November 2020), whereas Asia's contribution is largely confined to the first quarter. The monthly trend line chart illustrates how deaths consistently lag behind cases by approximately four weeks, evidencing reporting and clinical delays. Finally, the peak-incidence heatmap pinpoints emerging hotspots such as South Africa and the United Kingdom in late 2020, aligning with contemporaneous news reports. The animated choropleth highlights the geographical propagation of outbreaks, succinctly communicating temporal dynamics for presentations.

8.2 Performance Comparison

Table 4 summarises the observed runtimes averaged over three executions. MongoDB MapReduce benefits from running fully in memory and reusing the `mapReduce` engine, while Hadoop pays the startup cost of launching JVM containers. Spark executes in `local[4]` mode; despite a longer startup, it provides the richest API, Parquet outputs, and straightforward scaling to a cluster.

Table 4: Performance comparison across processing engines

Engine	Job	Runtime (s)	Notes
MongoDB	Country totals MR	0.35	Inline output; results exported to `analytics`
MongoDB	Peak 14-day MR	0.48	Computation handled in `finalize` phase
Hadoop Streaming	Country totals	1.82	Includes container startup overhead
Hadoop Streaming	Peak 14-day	2.46	Reducer sorts per-country entries in memory
Spark (local[4])	Aggregations	2.38	Writes three Parquet datasets; caches not used

Development effort differed markedly: MongoDB provided rapid iteration for exploratory analysis using the VS Code playground, Hadoop required more boilerplate (script deployment, HDFS staging), and Spark offered the most concise source code but incurred higher initial setup time (virtualenv, PySpark dependencies). For future operationalisation, running Spark on a managed cluster (Databricks, EMR, or Spark-on-K8s) would combine batch reliability with flexible APIs.

9 Discussion

The analysis confirms that pandemic burden was highly skewed: the United States alone contributed more cases than the bottom 150 countries combined, while Latin America exhibited consistently high CFRs. The rolling 14-day peaks highlight how quickly outbreaks escalated—for example, the United States exceeded 840k cases in a two-week window during December 2020. Limitations include under-reporting and inconsistent testing policies, particularly in early 2020, which explains the 31% of records lacking case counts and 59% lacking death counts. All computations treat missing values as zero, potentially underestimating totals; future iterations could impute from neighbouring dates. The dataset is aggregated at the national level, precluding intra-country analysis (e.g., states or counties). Nevertheless, the metrics remain useful for global situational awareness and resource allocation discussions.

The animated choropleth visually confirms the quantitative story: the outbreak radiates out of East Asia in Q1 2020, moves into Europe during March–April, and then dominates the Americas for the remainder of the year. This progression validates the aggregation results and provides an intuitive tool for communicating trends to non-technical stakeholders.

10 Conclusion and Next Steps

We successfully satisfied the Exercise 4 requirements: the dataset was profiled, key indicators were extracted, MapReduce programmes were implemented in both MongoDB and Hadoop, a Spark job produced equivalent aggregates, and visualisation-ready datasets were exported. MongoDB delivered the fastest turnaround for exploratory analysis, Hadoop ensured compatibility with legacy batch pipelines, and Spark provided a modern, scalable foundation for future enhancements.

Next steps include (i) automating the ingestion and analytics on a weekly schedule, (ii) extending the Spark job to compute additional metrics such as reproduction number proxies, and (iii) deploying the dashboards to a web portal for easy stakeholder access. Integrating hospital capacity data would enable richer correlations between case surges and healthcare utilisation.

Acknowledgements

I thank Prof. Nazim Agoulmine for guidance throughout the course and the ECDC for publishing the dataset under an open licence.

A Supplementary Material

A.1 Hadoop Job Commands

Concrete commands for running the Hadoop Streaming jobs are documented in `Hadoop/README.md`. The checklist covers HDFS staging, job submission, and result inspection.

A.2 MongoDB Analytics Collection

After executing `MongoDB/health_data.mongodb.js` the `analytics` collection contains four documents (metrics, payload arrays) which can be exported with `mongoexport -collection analytics -jsonArray`.

A.3 Python Visualisation Script

Run `visualizations/build_visualizations.py` to generate the static figures and animated choropleth discussed in Section 7. The script accepts CLI arguments for output folders and video filename (see inline usage example).

B Reproducibility Checklist

- **Repository structure.** All source code resides under `data_integration_and_big_-data/Project3`. MongoDB scripts (folder `MongoDB`), Hadoop streaming scripts (folder `Hadoop`), Spark job (folder `Spark`), and the LaTeX report (folder `Report`) are version controlled.
- **Data access.** Raw dataset: `Project3/data/health_data.json` (ECDC). Derived NDJSON: `Project3/data/encounters.ndjson`.
- **Execution.** MongoDB: run the playground in VS Code or execute `MongoDB/health_data.mongodb.js` with `mongosh`. Hadoop: follow `Hadoop/README.md`. Spark: submit `Spark/spark_health_analysis.py` with matching input/output arguments. Visualisations: run `visualizations/build_visualizations.py` with the dataset path.
- **Environment.** macOS Sonoma, MongoDB 7.0, Hadoop 3.3.6 (Streaming), Spark 3.5.0 with Python 3.10.
- **Report compilation.** `pdflatex report.tex` → `bibtex report` (bibliography optional) → `pdflatex` × 2.